UNIVERSITY OF RWANDA

RMA- GAKO

COLLEGE OF SCIENCE AND TECHNOLOGY

SCHOOL OF ICT

COMPUTER

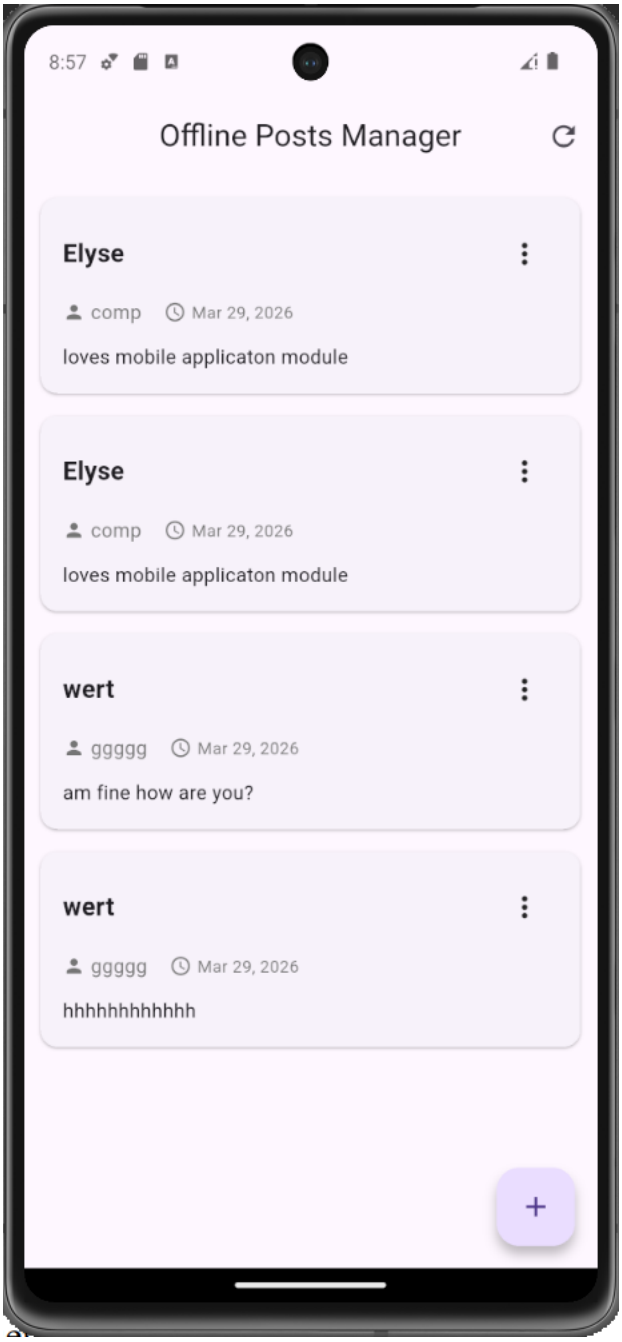
ENGINEERING LEVEL THREE

NAME: EDUARD GASASIRA

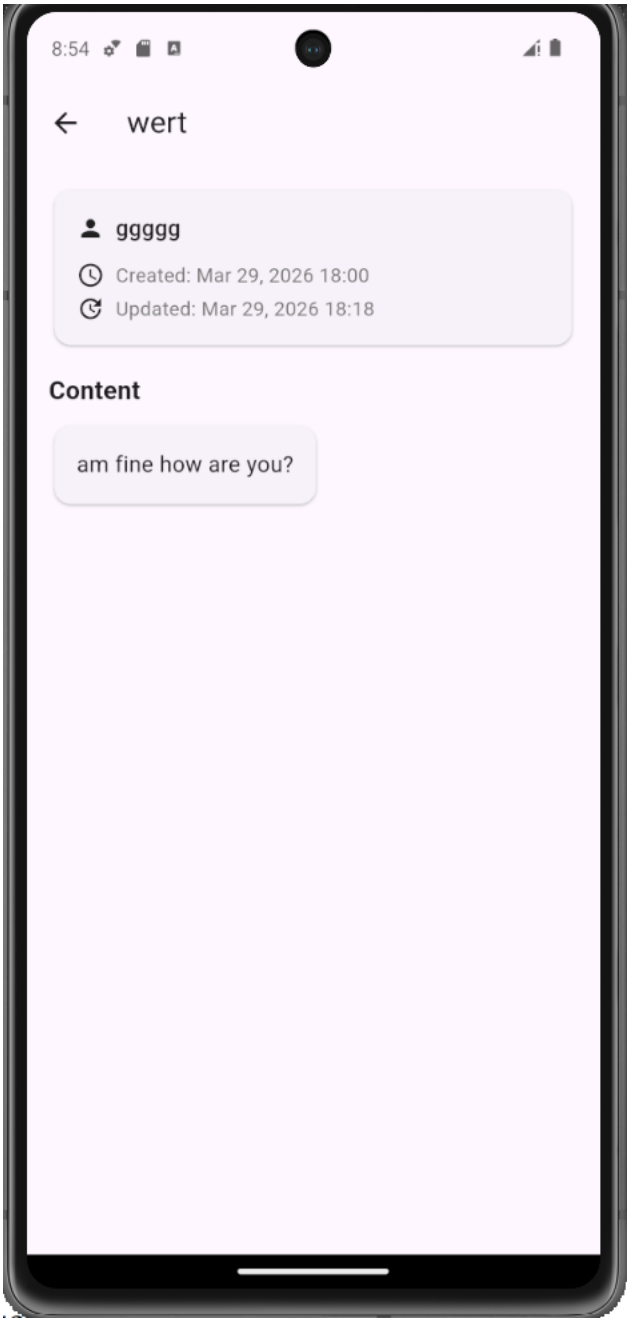
223026683

LAB 5: LOCAL STORAGE

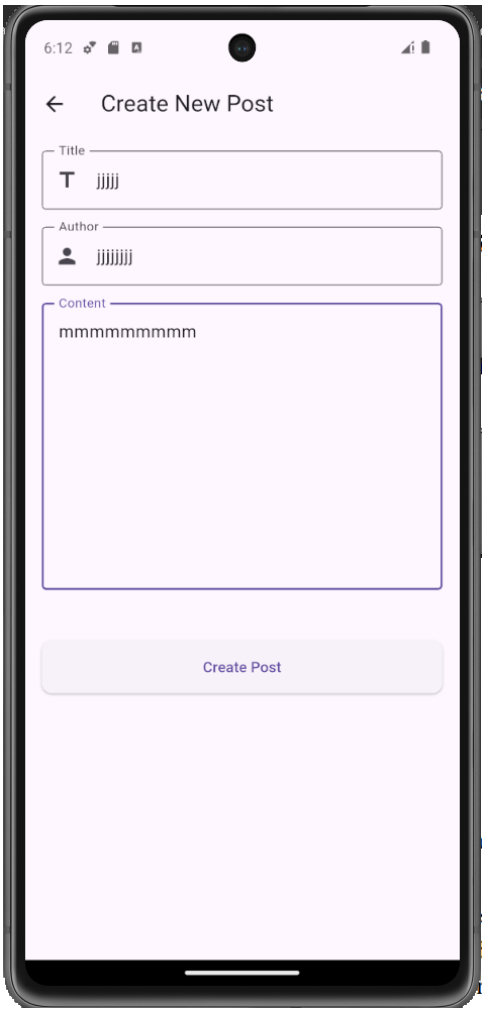
1. View all posts stored locally

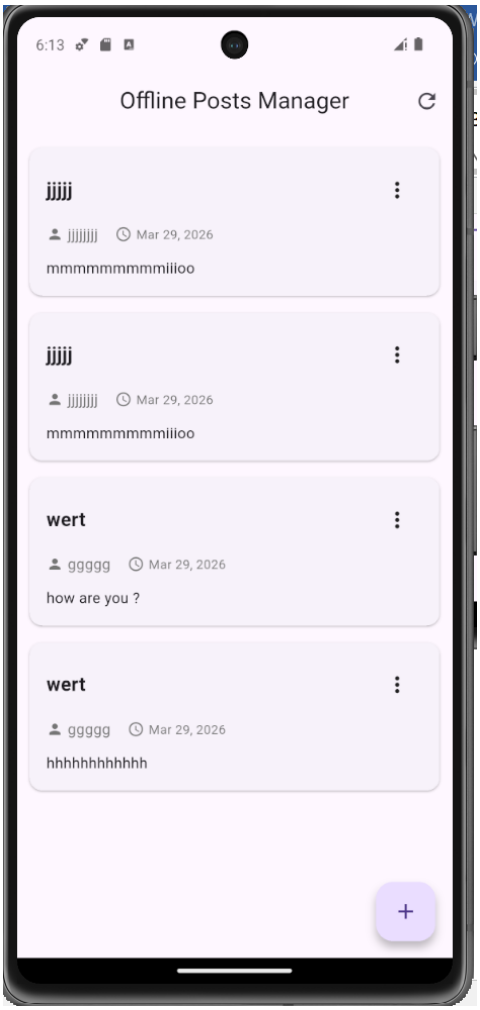


2. Read the details of a post

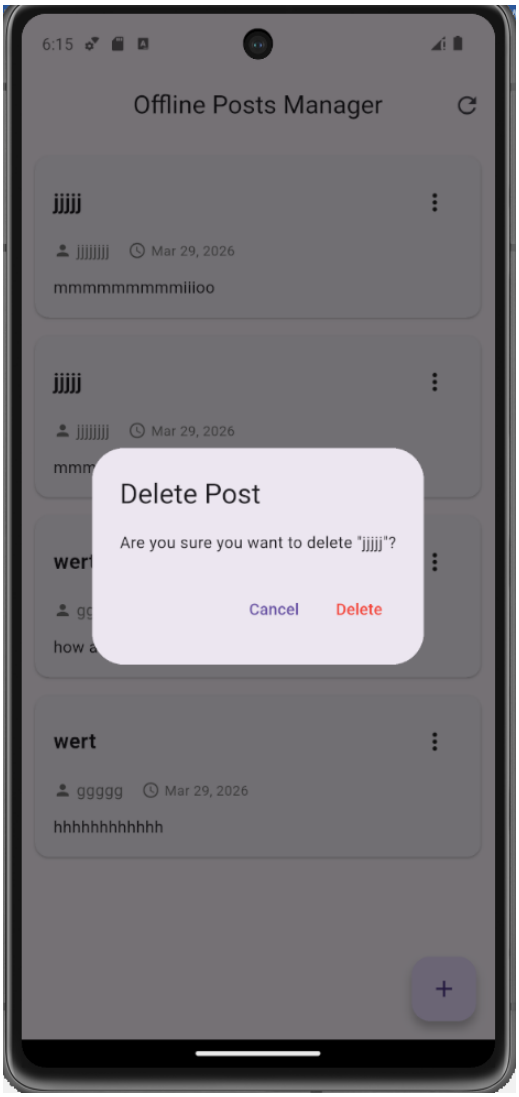


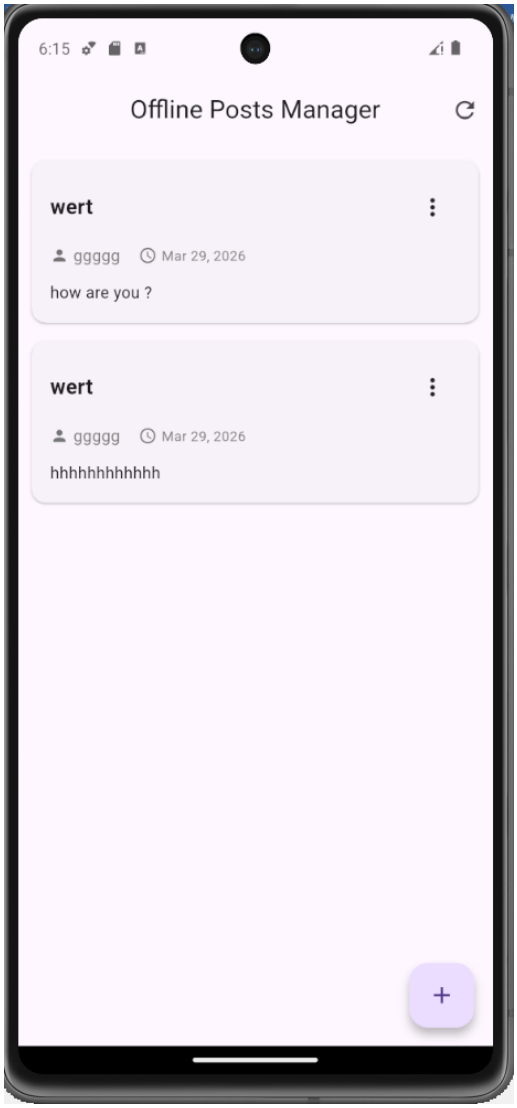
3. Add a new post



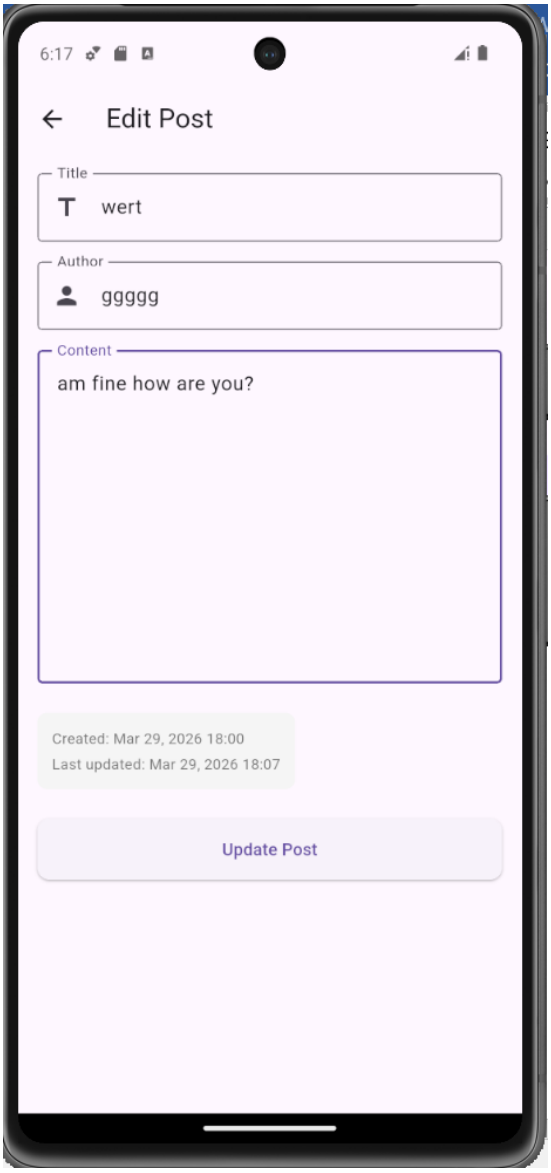


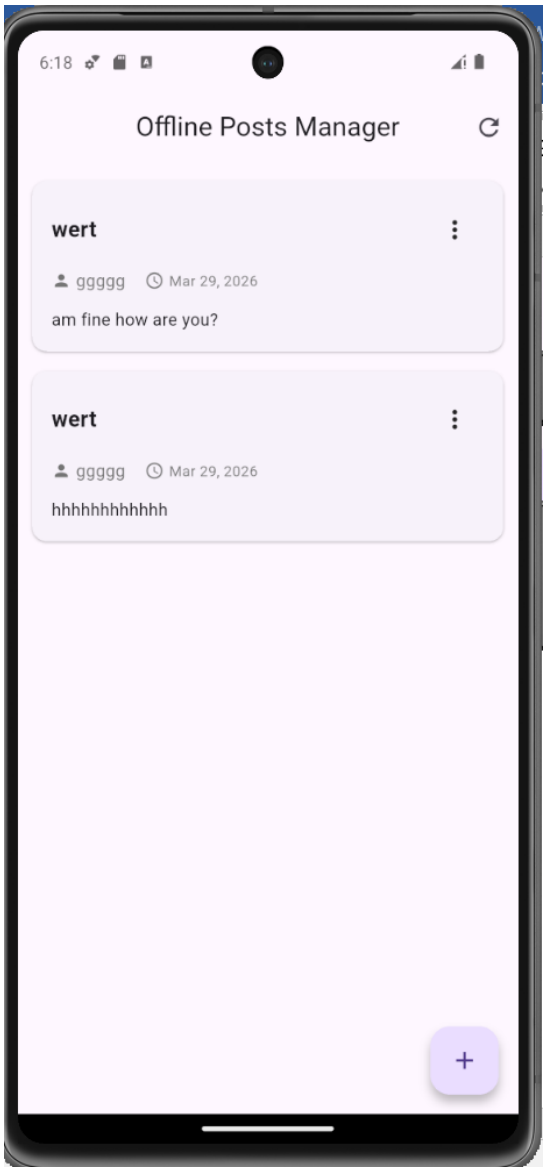
4. Delete a post


6:15
Offline Posts Manager
jjjjj
jjjjjjjj
Mar 29, 2026
mmmmmmmmmiiioo
jjjjj
jjjjjjjj
Mar 29, 2026
mmm
Delete Post
Are you sure you want to delete "jjjjj"?
Cancel
Delete
wert
how a
wert
ggggg
Mar 29, 2026
hhhhhhhhhhh


6:15
Offline Posts Manager
wert
ggggg
Mar 29, 2026
how are you ?
wert
ggggg
Mar 29, 2026
hhhhhhhhhhh

5. Edit an existing post





scanned handwritten materials:

LAB 5

1. Dependencies Used and why:

* Sqflite (^2.3.0)

This is the core dependency that enables SQLite database functionality in flutter. It provides a complete set of APIs for creating databases, defining tables, and performing all database operations. Without this plugin, flutter apps cannot interact with SQLite databases at all. It acts as the bridge between Dart code and the native SQLite engine on each platform.

* Sqflite_common_ffi (^2.3.0+1)

This dependency is specifically needed for desktop platforms like Windows, Linux, and macOS. It uses foreign function interface (FFI) to directly call SQLite's C library functions. This is essential for cross-~~platform function~~ compatibility because mobile devices handle sqlite differently than desktop operating systems. It ensures the same code works seamlessly across all platforms.

* Path (^1.8.3)

The path package provides utilities for manipulating file paths in a platform-independent way. Window uses backslashes, Unix uses forward slashes. This package handles these differences automatically, ensuring the database file is saved correctly on Android, iOS, Windows, and other platforms.

intl (^0.18.1)

The internationalization package is used for formatting dates in a human-readable format. When displaying, when a post was created or last updated, this package ensures dates are shown in a consistent, localized format across all devices.

Shared_preferences (^2.2.0)

This serves as an alternative storage mechanism for web platforms where SQLite has limitations. It stores data as key-value pairs in the browser's local storage, ensuring the app remains functional even in web browsers while maintaining offline capabilities.

✓ Why SQLite is Essential for Local Data Storage

* Persistent Data Storage

SQLite ensures that all posts created by staff members remain available even after the app is completely closed and reopened.

* Structured Data management

Unlike simple text files or shared preferences, SQLite allows for structured data organization with strict data types.

* Offline-first Architecture

SQLite enables a true offline-first approaches where the app remains fully functional without internet connectivity.

* Scalability

SQLite can handle databases up to 140 terabytes, far exceeding the needs ~~for~~ a typical post management systems.

Question 2 : Database Exception Handling

Handling Database Not Initialized

When the app first launches, the database must be properly initialized before any operations. The app uses a lazy initialization pattern where the database connection is only created when first needed.

example

```
Database? _database;
future <Database> get database async{
 if (_database != null) return _database!;
 _database = await initDatabase();
 return _database!;
}
```

Handling Insert/ update/ Delete errors

Before any write operation, the app performs comprehensive data validation. It checks that titles are at least 3 characters long, content exceeds 10 characters, and authors are not empty.

for Insert a null value into a NOT NULL column, are caught and translated into meaningful error messages. Staff members see specific feedback about what data needs correction rather than generic database errors.

for delete operations, the app first verifies that the post exists before attempting deletion.

ex:
```
try{
 await db.insert ('students', {'name': 'John'});
} catch (e) {
 print ("Insert error : $e");
```

Handling Invalid or Corrupted Data

The app validates each database record individually during reads, skipping corrupted posts while loading valid ones to prevent data loss. It automatically recovers salvageable data, corrects invalid timestamps, and logs issues for debugging without disrupting the user experience.

Example

Validate before inserting:

```
if (name.isEmpty) {
  throw Exception("invalid data");
}
```

Handle Corruption

```
try{
 await openDatabase(path);
} catch (e) {
  print ("Database corrupted, recreating...");
}
```

Question 3 : Sqlite in flutter Explained

✓ Database vs Table Distinction

A sqlite database is a single file that serves as a container for multiple related tables, much like an Excel workbook that contains multiple spreadsheets.

A table represents a specific collection of related data organized into rows and columns.

The database can contain multiple tables to represent different entities.

✓ CRUD Operations Explained

- Create Operation

When a staff member creates a new post, the app performs a create operation. This involves collecting the post data (title, content, author) and generating timestamps for creation and last update.

- Read operation

Reading data is the most frequent operation. The app reads all posts when loading the main screen, reading a single post when viewing details, and performing searches when staff members look for specific content.

- Update operations

When a staff member edits an existing post, the app performs an update operation. This requires identifying the specific post by its unique ID and modifying one or more fields.

- Delete Operations

Removing a post requires a delete operation. The app Confirms the user's intention through a dialog to prevent accidental deletions. When Confirmed, the app removes the post permanently from the database

✓ How flutter Interacts with Sqlite Asynchronously

Flutter interacts with sqlite asynchronously database operations can take time, especially with large datasets or slower devices. If these operations ran synchronously, they would freeze the entire user interface, causing the app to become unresponsive until the database operation Completed.

Example

```
future <void> insertStudent() async {
 final db = await database; // wait for database

 await db.insert ('students', {
   'name': 'John',
   'age': 20,
  });
 }
```

Key features

- async – marks a function as asynchronous
- await – waits for an operation to finish
- All database operations returns a future.